MessFinder

A Smart Mess Recommendation Platform for New Students

Final Capstone Project – Part I: Proposal and Prototyping

Ridhikanta Sarkar Sumit
Student ID: IT24045
Course: SEPM
Supervisor: Dr. Ziaur Rahman

October 30, 2026

Abstract

Every year, thousands of new students arrive on campus and must find a *mess* (a subscription-based meal service) that matches their budget, food habits and location. Today this is done through word of mouth, which is slow, unreliable and biased. **MessFinder** is a web platform where students enter their preferences and receive a ranked list of suitable messes, backed by menus and honest reviews from current students. Mess owners manage their listings and subscription requests, and an administrator verifies listings and moderates content. This document presents the proposal, schedule, UML models and a working prototype.

Contents

1 Introduction **3**
- 1.1 Problem Statement . . . 3
- 1.2 Objectives . . . 3
- 1.3 Scope . . . 3
- 1.4 Stakeholders . . . 3

2 Requirements **3**
- 2.1 Functional Requirements . . . 3
- 2.2 Non-Functional Requirements . . . 4
- 2.3 Recommendation Logic . . . 4

3 Feasibility and Risk Analysis **4**

4 Process Model, Schedule and Deliverables **4**
- 4.1 Chosen Process Model: Agile (Scrum) . . . 4
- 4.2 Sprint Plan . . . 5
- 4.3 Gantt Chart . . . 5
- 4.4 Milestones and Deliverables . . . 5

5 Resources and Tools **6**

6 System Design (UML and Related Diagrams) **6**
- 6.1 Data Flow Diagram . . . 6
- 6.2 Use Case Diagram . . . 7
- 6.3 Activity Diagram . . . 8
- 6.4 Entity Relationship Diagram . . . 9
- 6.5 Class Diagram . . . 10
- 6.6 Object Diagram . . . 11

7 Prototype **11**
7.1 Screenshots 12
7.2 Class-to-Code Mapping 14

8 Testing Plan **14**

9 Conclusion **14**

1 Introduction

1.1 Problem Statement

New students have no reliable way to compare messes. Information about price, distance, hygiene and menu quality is scattered across personal contacts and social media posts. As a result students often choose poorly, pay more than necessary, or must switch after a month, while good small messes struggle to reach new customers.

1.2 Objectives

1. Provide a single searchable directory of verified messes near campus.
2. Rank messes automatically by how well they match each student's preferences.
3. Show weekly menus, prices and multi-criteria reviews (food, hygiene, service, value).
4. Let students request a subscription and owners approve or reject it online.
5. Give administrators tools to verify listings and moderate reviews.

1.3 Scope

In scope: user accounts for three roles, mess listing and menu management, search/filter, preference-based ranking, reviews and ratings, subscription requests, admin moderation, a responsive web interface.
Out of scope (future work): online payment, delivery tracking, native mobile apps, AI-based menu analysis.

1.4 Stakeholders

Stakeholder	Interest
Students	Find a suitable, affordable, hygienic mess quickly.
Mess owners	Reach more customers; manage menu, prices and subscribers.
Administrator	Keep listings genuine and reviews fair.
University / hall authority	Better student welfare and food-quality visibility.
Project supervisor	Evaluate scope, quality and documentation.

2 Requirements

2.1 Functional Requirements

ID	Requirement
FR1	Users shall register and log in as Student, Mess Owner or Admin.
FR2	Students shall set preferences: maximum budget, maximum distance, food type, meals per day.
FR3	The system shall search and filter messes and rank them by a match score.
FR4	The system shall display mess details and the weekly menu.
FR5	Students shall compare two or more messes side by side.
FR6	Students shall submit reviews with ratings for food, hygiene, service and value.
FR7	Students shall request a subscription; owners shall approve or reject it.
FR8	Owners shall create and edit their mess profile, menu and prices.
FR9	Admins shall approve new mess listings and remove inappropriate reviews.

2.2 Non-Functional Requirements

Attribute	Requirement
Performance	Search results shall load in under 2 seconds for up to 500 messes.
Usability	A new student shall find a mess within 3 minutes without training; mobile-friendly layout.
Security	Passwords hashed; role-based access control; input sanitised against XSS and SQL injection.
Reliability	Availability of at least 99% during admission season.
Maintainability	Modular, object-oriented code that mirrors the class diagram.

2.3 Recommendation Logic

Each approved mess receives a match score out of 100 for a student's preferences:

$$\text{Score} = S_{price}(35) + S_{distance}(25) + S_{food}(20) + S_{meals}(10) + \tfrac{\text{rating}}{5} \times 10$$

Price and distance earn full points within the student's limits and lose points proportionally beyond them. Food type earns points on an exact match (or "Any"), meals on meeting the requested count.

3 Feasibility and Risk Analysis

Technical: uses mature, free technologies. **Economic:** negligible cost; free hosting tiers suffice for the pilot. **Operational:** owners need only a simple form; students use a browser.

Risk	Level	Mitigation
Few owners join at launch	High	Personally onboard 10–15 nearby messes; admin can enter listings on their behalf.
Fake or biased reviews	Medium	Only students may review; admin moderation; one review per student per mess.
Scope creep	Medium	Agile backlog with strict sprint goals; extras go to future work.
Outdated menus/prices	Medium	Show "last updated" date; reminders to owners.
Schedule slip	Medium	Buffer in final sprint; prioritise must-have features first.

4 Process Model, Schedule and Deliverables

4.1 Chosen Process Model: Agile (Scrum)

Requirements such as filters and ranking weights will change after students try the system, so an iterative, feedback-driven model fits better than Waterfall. The project runs as **six two-week sprints (12 weeks)**, each ending with a working increment and a review.

4.2 Sprint Plan

Sprint	Weeks	Goal
0	1–2	Requirements, proposal, UML design, prototype
1	3–4	Database schema, authentication, mess listing and menu management
2	5–6	Search, filters, preference-based ranking, comparison view
3	7–8	Reviews and ratings, subscription workflow
4	9–10	Admin moderation, UI polish, system testing
5	11–12	User acceptance test with students, deployment, final report

4.3 Gantt Chart

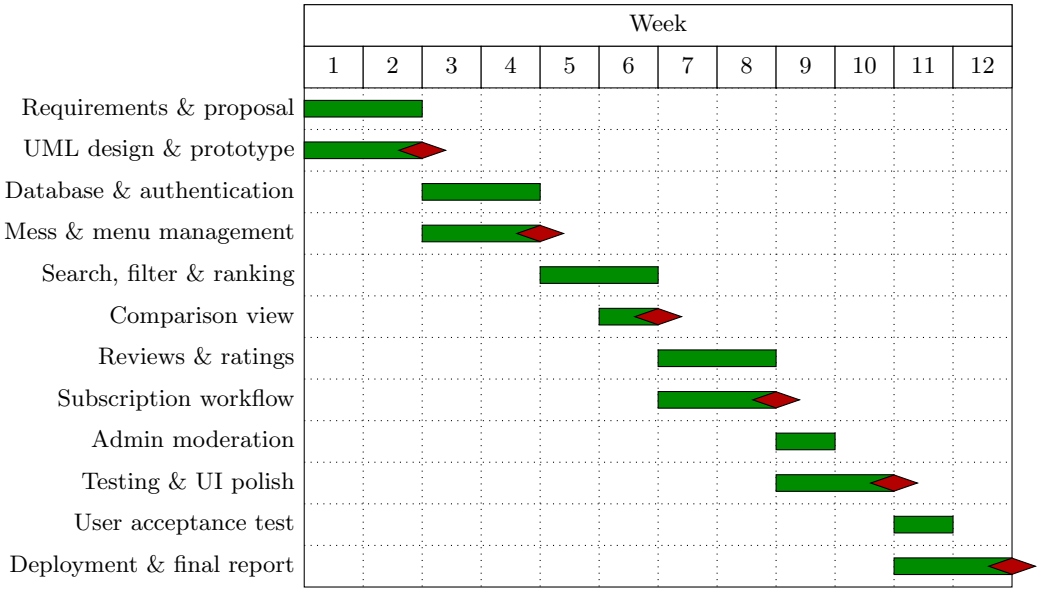


Figure 1: Project Gantt chart (12 weeks, 6 sprints; red markers are milestones M1–M6, listed in the table below).

4.4 Milestones and Deliverables

ID	Week	Milestone	Deliverables
M1	2	Proposal and design approved	Proposal, UML diagrams, Gantt chart, prototype
M2	4	Core data and accounts ready	Database schema, login, mess/menu CRUD
M3	6	Recommendation engine working	Search, filters, ranking, comparison page
M4	8	Social and booking features	Reviews, ratings, subscription workflow
M5	10	System tested	Admin tools, test report, bug fixes
M6	12	Final release	Deployed system, user manual, final report, demo

5 Resources and Tools

Team	1 student developer (analyst, designer, developer, tester); supervisor as product owner.
Technology	Python 3, Flask/Django (full system), SQLite then MySQL/PostgreSQL, HTML/CSS/JS.
Design	Graphviz / draw.io for UML; LaTeX for documentation.
Management	Git and GitHub (version control, issues as backlog), GanttProject / pgfgantt.
Budget	Approx. 0 BDT development cost (free tiers); optional domain & hosting about 1,500 BDT/year.

6 System Design (UML and Related Diagrams)

6.1 Data Flow Diagram

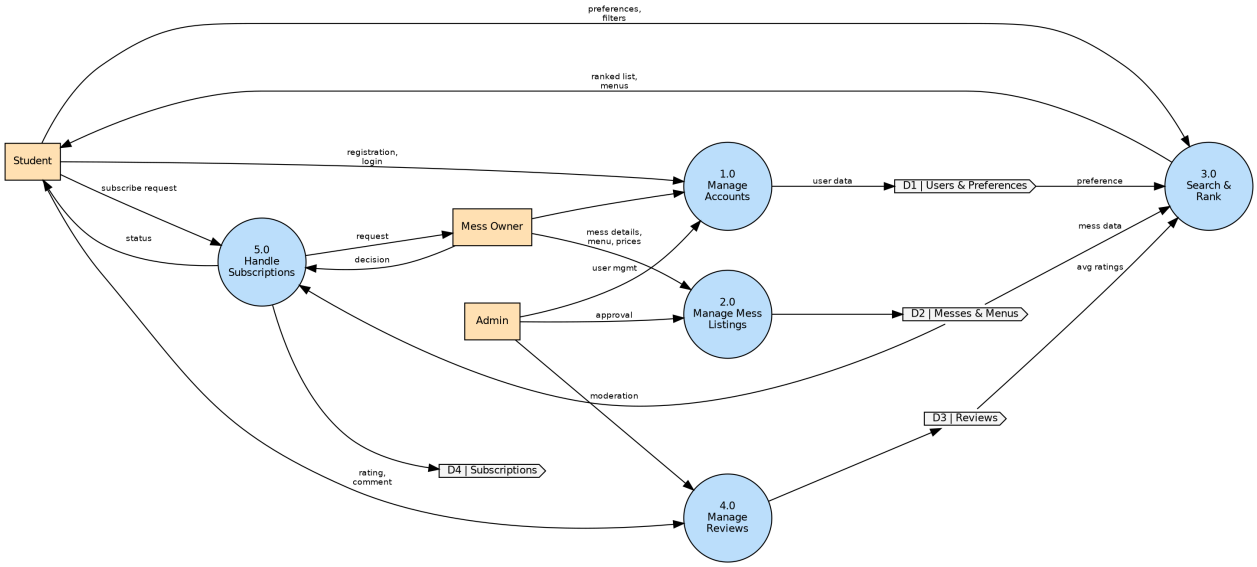


Figure 2: Level-1 Data Flow Diagram: five processes, four data stores.

6.2 Use Case Diagram



Figure 3: Use case diagram for Student, Mess Owner and Admin.

6.3 Activity Diagram

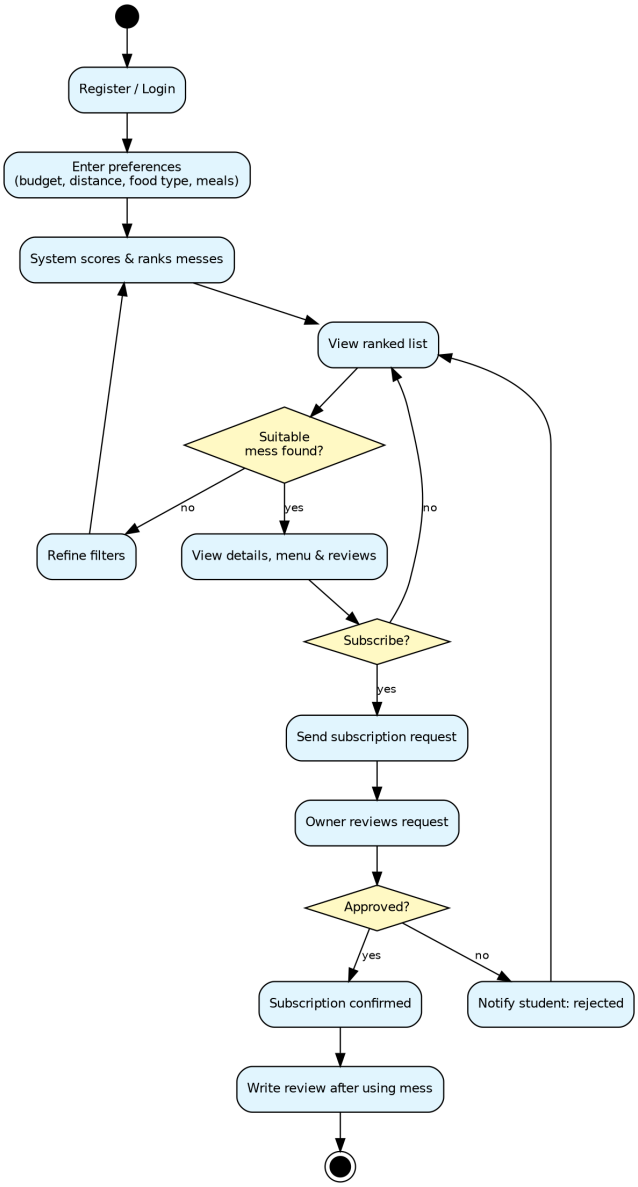


Figure 4: Activity diagram: student finds and subscribes to a mess.

6.4 Entity Relationship Diagram

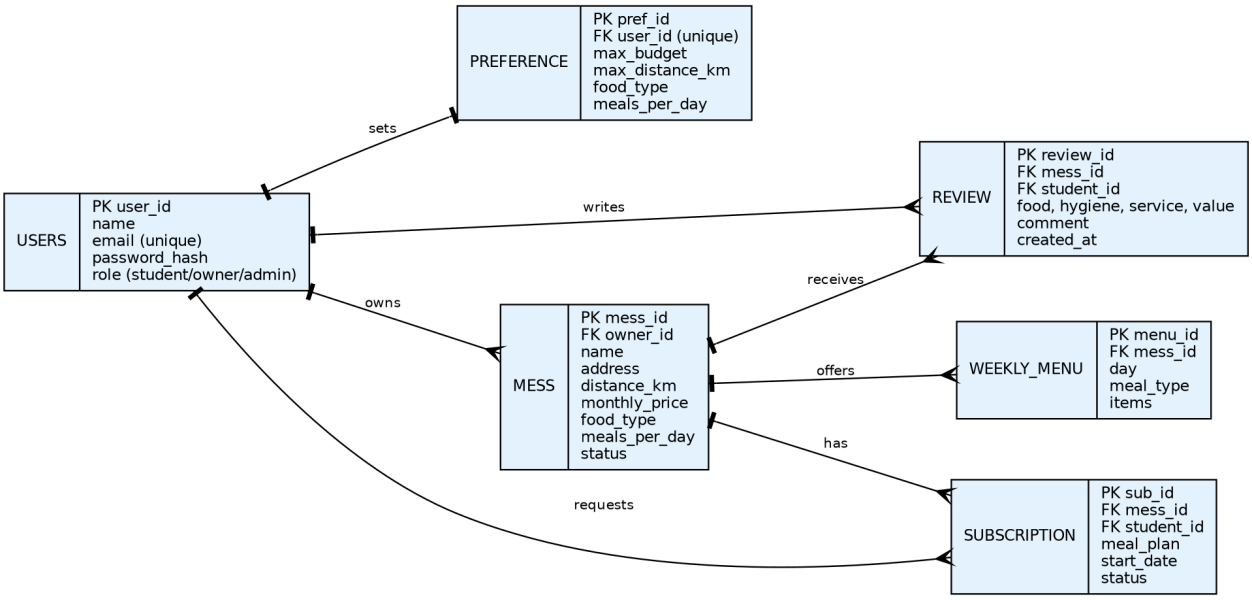


Figure 5: ER diagram of the MessFinder database (crow's foot notation).

6.5 Class Diagram

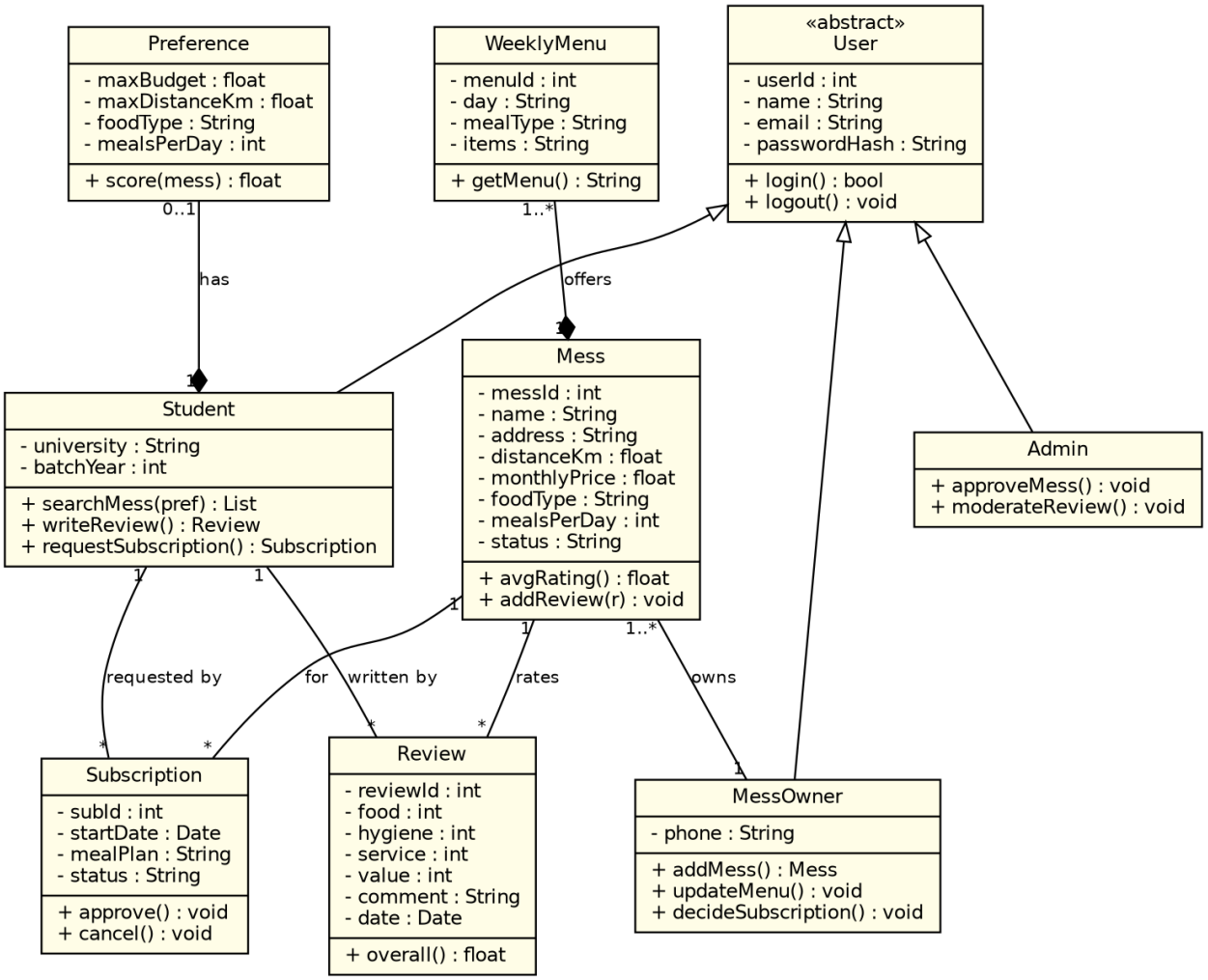


Figure 6: Class diagram: 9 classes with inheritance, composition and associations.

6.6 Object Diagram

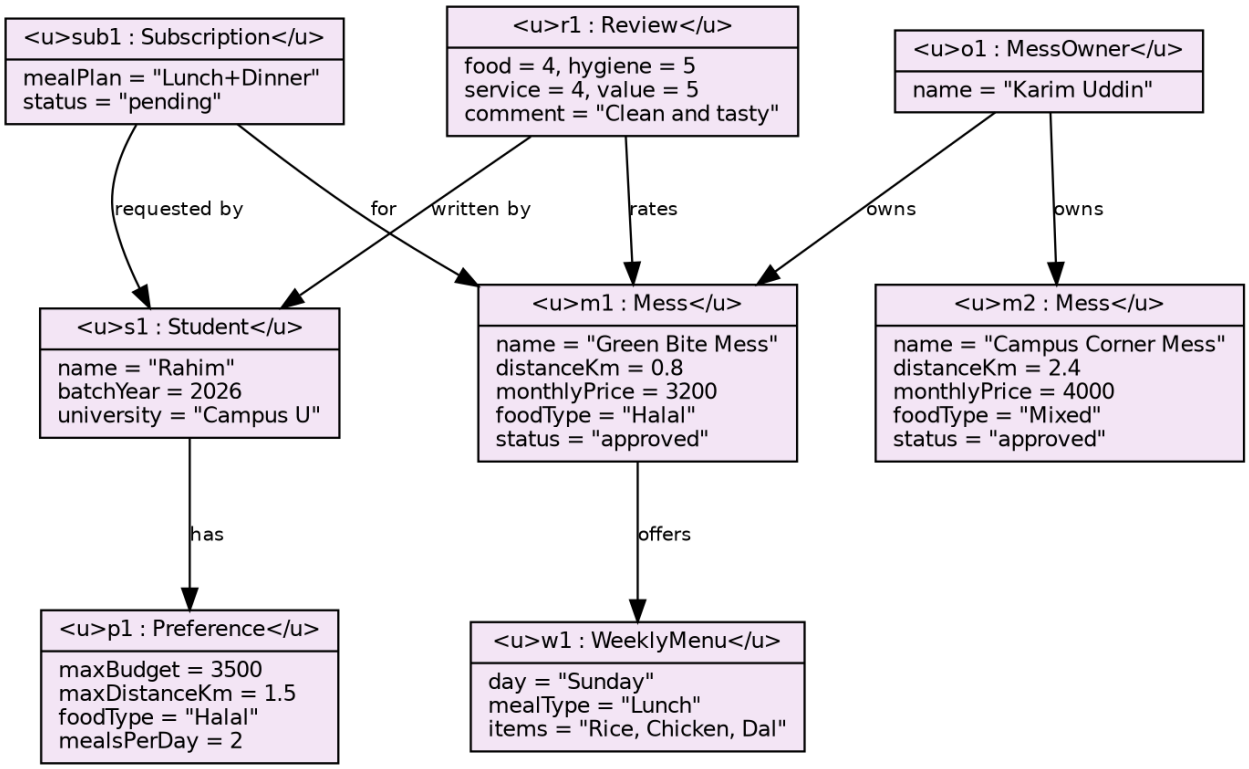


Figure 7: Object diagram: a snapshot of runtime objects for one student's search.

7 Prototype

The prototype (folder `prototype/`) is a working web interface written in Python (standard library only). It implements nine classes from the class diagram in `models.py`: `User`, `Student`, `MessOwner`, `Admin`, `Mess`, `WeeklyMenu`, `Review`, `Subscription` and `Preference`. Run `python3 app.py` and open `http://localhost:8000`.

- **Home:** preference form and ranked list with match percentage.
- **Mess page:** details, weekly menu, reviews, review form, subscription request.
- **Demo approval link:** simulates the owner approving a request.

7.1 Screenshots

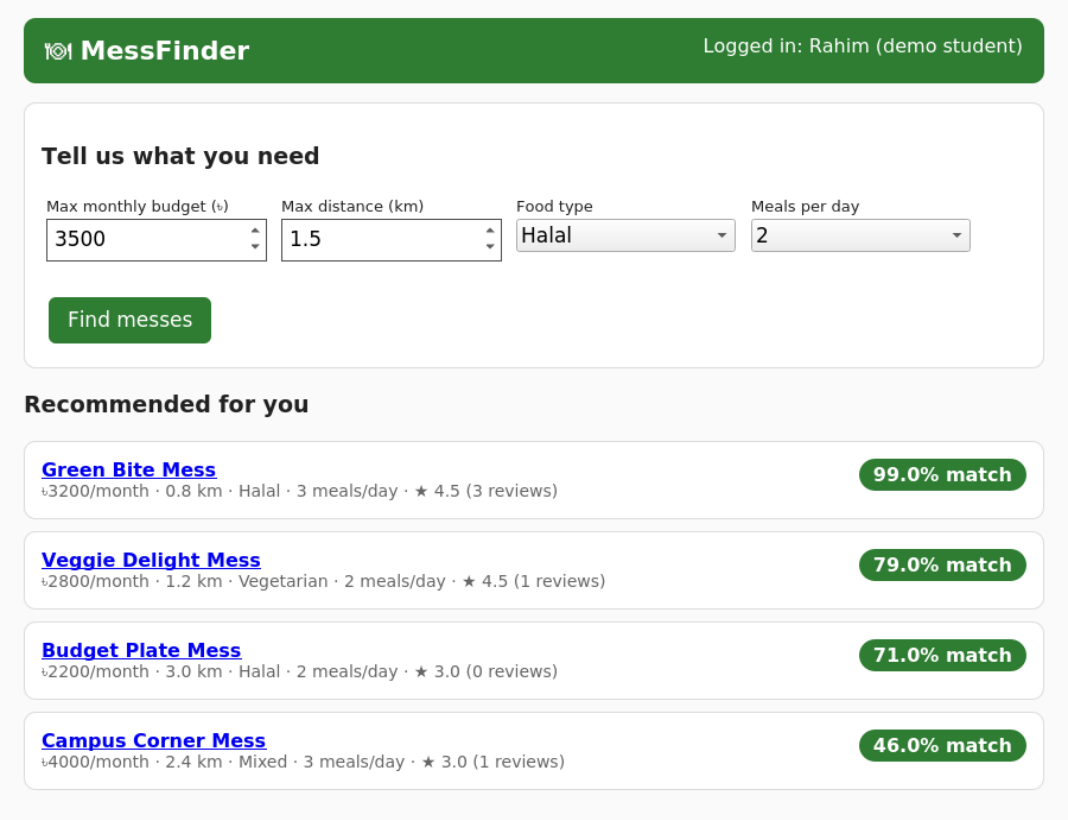


Figure 8: Home page: preference form and ranked recommendations with match percentage.

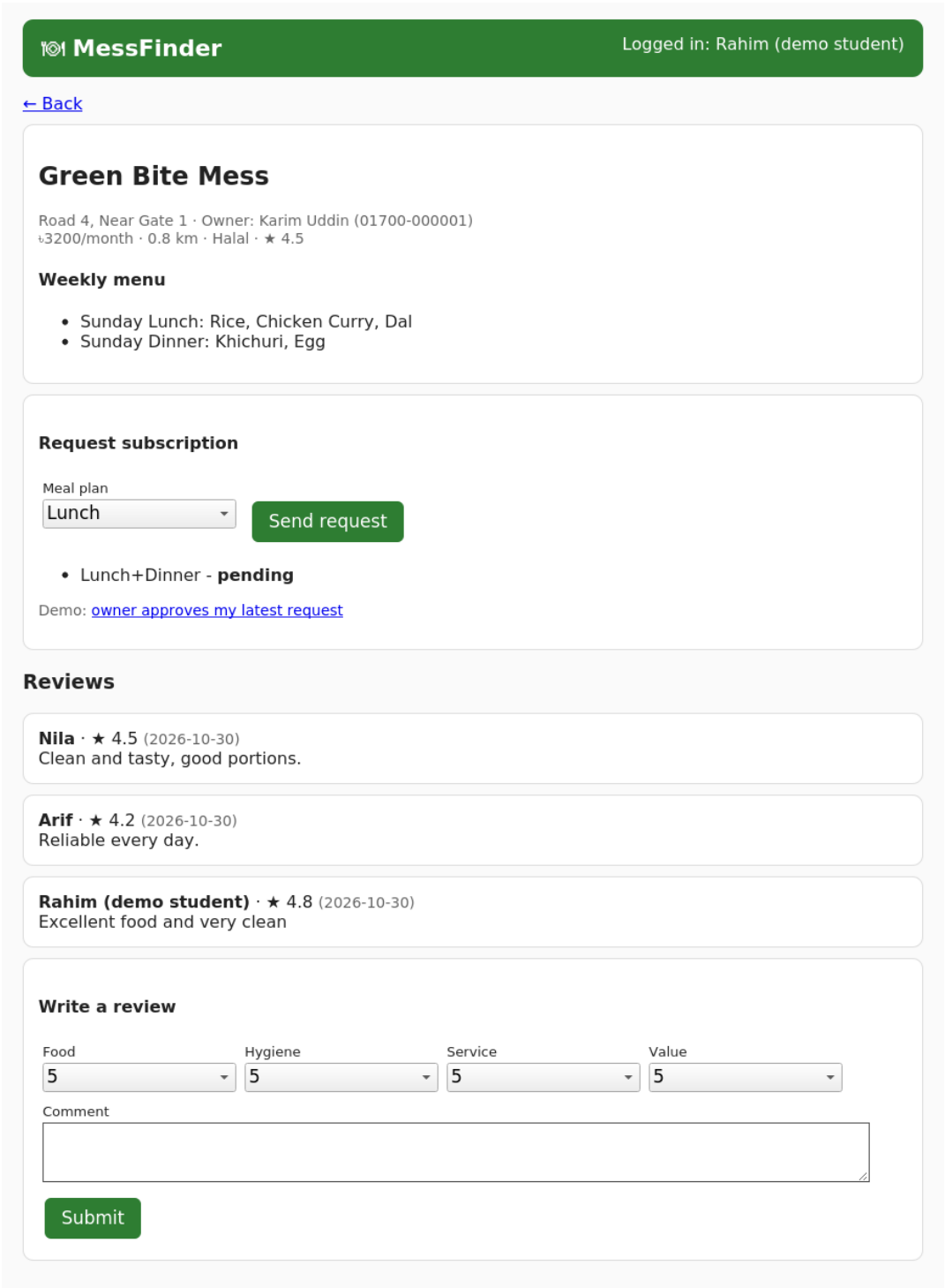


Figure 9: Mess page: details, weekly menu, subscription request and reviews.

7.2 Class-to-Code Mapping

Class	Implemented behaviour in `models.py`
User, Student, MessOwner, Admin	Inheritance; `login()`, `search_mess()`, `write_review()`, `request_subscription()`, `add_mess()`, `decide_subscription()`, `approve_mess()`, `moderate_review()`
Preference	`score(mess)` – the 0–100 match score
Mess	`avg_rating()`, `add_review()`
WeeklyMenu	`get_menu()`
Review	`overall()` – mean of four ratings
Subscription	`approve()`, `cancel()`

8 Testing Plan

Ten automated unit tests already exist in `prototype/test_models.py` (run `python3 -m unittest -v`; all pass) covering scoring, ranking, reviews, subscriptions and admin actions. Planned next: integration tests for the web workflows; usability testing with at least 10 new students; and security checks for input handling and role access.

9 Conclusion

MessFinder solves a real, recurring problem for new students with a feasible scope, a clear Agile plan and a design fully modelled in UML. The prototype already demonstrates the core idea: preference-based ranking of messes backed by menus, reviews and subscriptions.

Repository: https://github.com/sumitnoob/SEPM-project